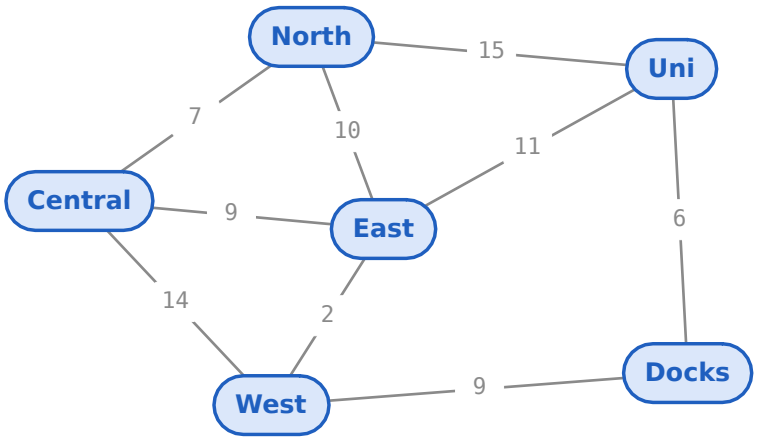
Chapter 21

Graphs and dynamic programming

most hard problems are a graph or a table in disguise

A **graph** is things (nodes) and connections (edges): branches and courier routes, web pages and links, modules and prerequisites, people and friendships. Once a problem is drawn as a graph, a handful of algorithms answer most questions about it. Our example: six library branches and the minutes a courier needs between them.



undirected: routes work both ways

weighted: minutes on every edge

6 nodes, 9 edges

```python
from collections import defaultdict

ROUTES = '''Central North 7   Central East 9   Central West 14
            North East 10     North Uni 15     East West 2
            East Uni 11       West Docks 9     Uni Docks 6'''
words = ROUTES.split()
routes = [(a, b, int(m)) for a, b, m in zip(words[::3], words[1::3], words[2::3])]

graph = defaultdict(list)                # adjacency list: node -> [(neighbour, minutes)]
for a, b, minutes in routes:
    graph[a].append((b, minutes))
    graph[b].append((a, minutes))        # undirected: store both directions
print(graph["East"])
```

```
[('Central', 9), ('North', 10), ('West', 2), ('Uni', 11)]
```

store it as	space	neighbours of u	is u–v an edge?	best for
adjacency list (dict of lists)	V + E	O(degree)	O(degree)	almost everything
adjacency matrix (V×V grid)	V^2	O(V)	O(1)	dense graphs, NumPy maths
edge list	E	O(E)	O(E)	input files, sorting edges

Breadth-first search: fewest steps

BFS explores in rings: first every neighbour of the start, then their neighbours, and so on. A **queue** (first in, first out) keeps the rings in order, so the first time BFS reaches a node is by the **fewest edges**:

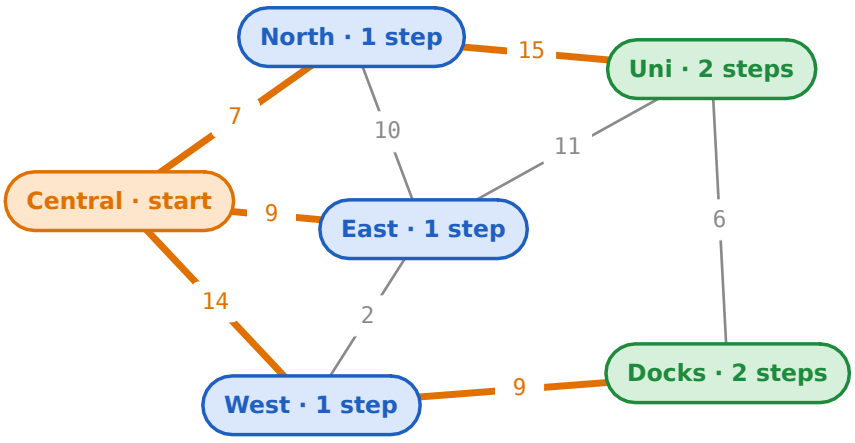
```python
from collections import deque

def bfs(graph, start):
    dist, prev = {start: 0}, {}
    queue = deque([start])
    while queue:
        u = queue.popleft()
        for v, _ in graph[u]:
            if v not in dist:               # first visit = fewest steps
                dist[v], prev[v] = dist[u] + 1, u
                queue.append(v)
    return dist, prev

def path(prev, end):
    out = [end]
    while out[-1] in prev:                  # walk the 'came from' links back
        out.append(prev[out[-1]])
    return out[::-1]

dist, prev = bfs(graph, "Central")
print(dist)
print(path(prev, "Docks"))
```

```
{'Central': 0, 'North': 1, 'East': 1, 'West': 1, 'Uni': 2, 'Docks': 2}
['Central', 'West', 'Docks']
```



thick: how BFS first reached each node

colour: steps from Central

KEY IDEA

BFS ignores the weights: Central → West → Docks is 2 steps but 23 minutes, while the quickest route has 3 steps. Each node and edge is handled once: O(V + E). BFS is the right tool whenever every step costs the same: mazes, word ladders, degrees of separation, the level-order walk of Ch 20.

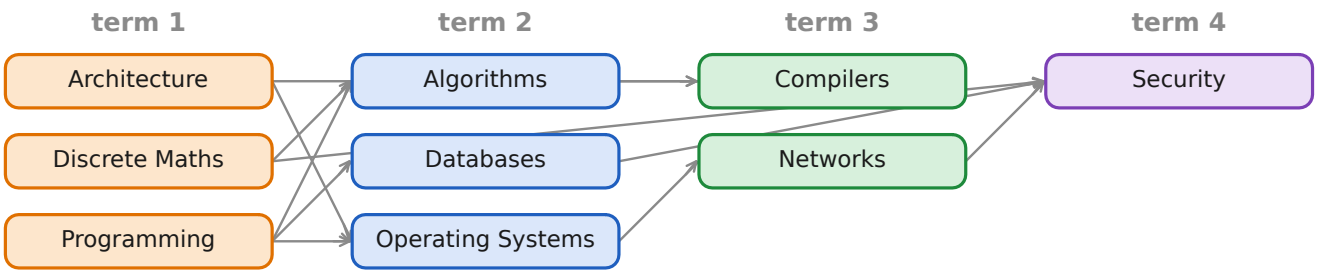
Depth-first search and a safe order of work

Swap the queue for a **stack** and the search goes as deep as it can before backing up. That's DFS: the tool for reachability, cycles and ordering. Module prerequisites are a **directed** graph (an edge means 'needs'), and a valid study plan is a **topological order**: every module after the ones it needs. The stdlib's `graphlib` does it, and can even hand out modules term by term:

```
1  from graphlib import TopologicalSorter, CycleError
2
3  needs = {                                          # module: the modules it needs first
4      "Algorithms": ["Programming", "Discrete Maths"],
5      "Databases": ["Programming"],
6      "Operating Systems": ["Architecture", "Programming"],
7      "Networks": ["Operating Systems"],
8      "Security": ["Networks", "Databases", "Discrete Maths"],
9      "Compilers": ["Algorithms", "Architecture"],
10 }
11 ts = TopologicalSorter(needs)
12 ts.prepare()
13 terms = []
14 while ts.is_active():
15     ready = sorted(ts.get_ready())                 # everything whose needs are all done
16     terms.append(ready)
17     ts.done(*ready)
18 for k, ready in enumerate(terms, 1):
19     print(f"term {k}: {', '.join(ready)}")
```

```
term 1: Architecture, Discrete Maths, Programming
term 2: Algorithms, Databases, Operating Systems
term 3: Compilers, Networks
term 4: Security
```

line 12: raises CycleError if there's a loop



```python
def depends_on(graph, start):
    seen, stack = set(), [start]
    while stack:
        u = stack.pop()                          # a stack: go deep first
        for v in graph.get(u, []):
            if v not in seen:
                seen.add(v)
                stack.append(v)
    return seen

print(sorted(depends_on(needs, "Networks")))
needs["Programming"] = ["Security"]              # a circular requirement
try:
    list(TopologicalSorter(needs).static_order())
except CycleError as e:
    print("cycle:", " -> ".join(e.args[1]))
```

```
['Architecture', 'Operating Systems', 'Programming']
cycle: Programming -> Databases -> Security -> Programming
```

WHY

Topological order exists only when there is no cycle (a DAG: directed acyclic graph). The same idea orders build steps (make), package installs (pip's resolver, Ch 19), spreadsheet cells and database migrations. A DFS-based version is practice 21.2.

Dijkstra: cheapest paths with a heap

With weights, the fewest steps isn't the quickest route. Dijkstra's algorithm grows a set of **settled** nodes whose best time is final, always settling the unsettled node with the smallest known time next. A heap (Ch 10) finds that node in O(log n):

```python
import heapq

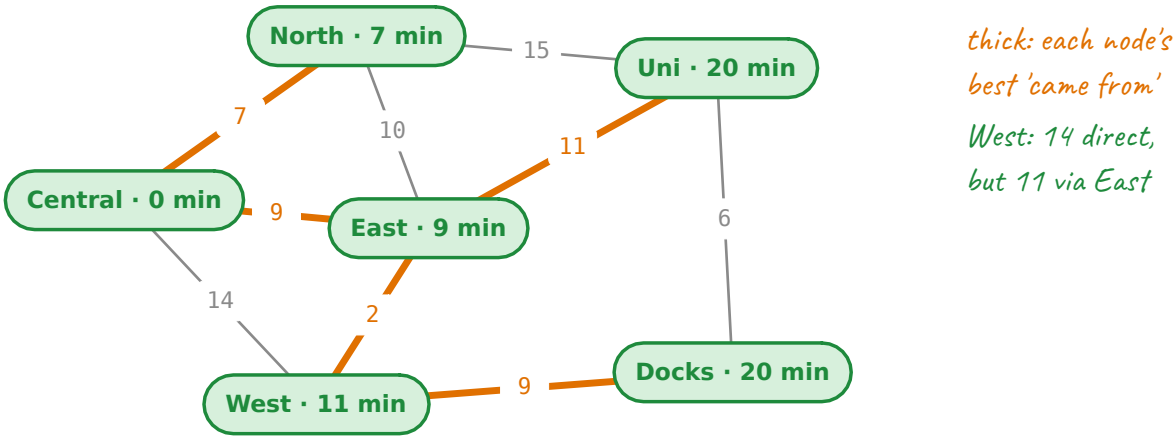
def dijkstra(graph, start):
    dist, prev = {start: 0}, {}
    heap = [(0, start)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue                     # stale: u was already settled more cheaply
        for v, minutes in graph[u]:
            new = d + minutes
            if new < dist.get(v, float("inf")):
                dist[v], prev[v] = new, u
                heapq.heappush(heap, (new, v))
    return dist, prev

best, came_from = dijkstra(graph, "Central")
print(path(came_from, "Docks"), best["Docks"], "minutes")
```

```
['Central', 'East', 'West', 'Docks'] 20 minutes
```

line 12: found a quicker way to v

popped	action	improved (pushed)
(0, Central)	settle Central at 0	North 7, East 9, West 14
(7, North)	settle North at 7	Uni 22
(9, East)	settle East at 9	West 11, Uni 20
(11, West)	settle West at 11	Docks 20
(14, West)	stale: skip it	(West was settled at 11)
(20, Docks)	settle Docks at 20	nothing better
(20, Uni)	settle Uni at 20	nothing better
(22, Uni)	stale: skip it	(Uni was settled at 20)



TRAP

Dijkstra is only correct with **non-negative** weights: it assumes a settled node can't get cheaper later. With negative edges use Bellman-Ford (O(V·E)). The `if d > dist[u]: continue` line is the price of not having a decrease-key operation on `heapq`: old entries are skipped when they surface. Cost: O((V + E) log V).

Dynamic programming: from recursion to a table

Dynamic programming is Ch 20's memoisation, applied on purpose: describe the answer in terms of answers to **smaller versions** of the same question, and make sure each smaller answer is computed once. How many ways can you pay 100p with 1p, 2p and 5p coins?

top-down: recursion + cache

```python
from functools import cache

COINS = (1, 2, 5)

@cache
def count(amount, i=0):
    if amount == 0:
        return 1
    if amount < 0 or i == len(COINS):
        return 0
    use = count(amount - COINS[i], i)
    skip = count(amount, i + 1)
    return use + skip

print(count(100))
```

```
541
```

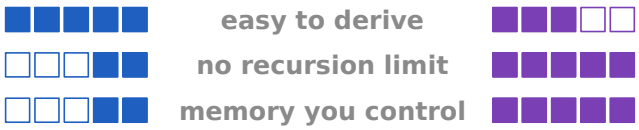
bottom-up: fill a table

```python
COINS = (1, 2, 5)

ways = [1] + [0] * 100     # index = amount
for c in COINS:
    for a in range(c, 101):
        ways[a] += ways[a - c]

print(ways[100])
```

```
541
```



step	question to answer	coins example
1. state	what does a subproblem need to know?	amount left, which coins are still allowed
2. recurrence	how do smaller answers combine?	use coin i again + skip coin i
3. base cases	which answers are known outright?	amount 0: 1 way; no coins left: 0 ways
4. order	what must be computed before what?	smaller amounts first (the table's loop order)
5. answer	which cell holds the result?	`count(100)` / `ways[100]`

A DP table you can see: edit distance

How many single-letter edits turn one word into another? It's the typosquat alarm of Ch 19 and every spell checker. Each cell answers a smaller question, and the table fills row by row:

```python
def edit_table(a, b):
    d = [[0] * (len(b) + 1) for _ in range(len(a) + 1)]
    for i in range(len(a) + 1):
        d[i][0] = i                                  # delete all i letters
    for j in range(len(b) + 1):
        d[0][j] = j                                  # insert all j letters
    for i in range(1, len(a) + 1):
        for j in range(1, len(b) + 1):
            d[i][j] = min(d[i - 1][j] + 1,                              # delete a[i-1]
                          d[i][j - 1] + 1,                              # insert b[j-1]
                          d[i - 1][j - 1] + (a[i - 1] != b[j - 1]))  # keep/replace
    return d

d = edit_table("kitten", "sitting")
print(d[-1][-1])
```

```
3
```

		s	i	t	t	i	n	g
	0	1	2	3	4	5	6	7
k	1	1	2	3	4	5	6	7
i	2	2	1	2	3	4	5	6
t	3	3	2	1	2	3	4	5
t	4	4	3	2	1	2	3	4
e	5	5	4	3	2	2	3	4
n	6	6	5	4	3	3	2	3

cell [i][j] = fewest edits to turn the first i letters of a into the first j letters of b

↘ *keep (free) or replace (+1)*
↓ *delete from a (+1)*
→ *insert from b (+1)*

answer: bottom-right = 3

Following the orange cells back from the corner recovers the edits themselves: replace k→s, keep i, t, t, replace e→i, keep n, insert g. The table costs O(len(a) × len(b)) time and memory; keeping only the previous row cuts the memory to O(len(b)).

When greedy is enough, and when it isn't

```python
COINS = [4, 3, 1]

def greedy(amount):                          # always take the biggest coin that fits
    used = []
    for c in COINS:
        while amount >= c:
            amount -= c
            used.append(c)
    return used

def fewest(amount):                          # DP: best[a] = fewest coins that make a
    best = [[]] + [None] * amount
    for a in range(1, amount + 1):
        best[a] = min((best[a - c] + [c] for c in COINS if c <= a), key=len)
    return best[amount]

print(greedy(6), fewest(6))
```

```
[4, 1, 1] [3, 3]
```

KEY IDEA

A **greedy** algorithm commits to the best-looking choice and never reconsiders. It's right only when you can prove the local choice is safe: Dijkstra is greedy (safe because weights are non-negative), and so is paying with UK coins. When choices interact, as with coins 4, 3, 1, DP tries every option for every subproblem, once each.

Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 21.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 21.1` shows other valid solutions and their trade-offs.

21.1 build from a spec Fewest steps through a maze
21.2 build from a spec Your own topological sort
21.3 fix the bug Dijkstra that settles too early
21.4 learn an unseen module Islands in a random network
21.5 build from a spec Longest common subsequence
21.6 refactor, tests stay green Packing the book bag: knapsack by DP
21.7 mini-project A courier route planner

PASSPORT STAMP ask the same question of every new language (Appendix B)

6. Data structures

Graphs as `dict` of neighbour lists (or `defaultdict(list)`); `deque` for BFS, a list as a stack for DFS, `heapq` for Dijkstra. DP tables are lists of lists, or `@cache` on a recursive function.

12. Stdlib and tooling

`graphlib.TopologicalSorter` (ordering, cycle detection), `heapq`, `collections.deque`, `functools.cache`. For serious graph work: `networkx` from pip (Ch 19).

IN OTHER LANGUAGES

Every language has the parts: C++ `std::priority_queue` and `std::queue`, Java `PriorityQueue` and `ArrayDeque`, Rust `BinaryHeap` (a max-heap: wrap keys in `Reverse`) and `VecDeque`. C has none built in, so a graph is arrays of indices. The algorithms are identical; only the containers' names change.

DEEP DIVE (your vault)

Vault notes: 13.13 Graphs · 13.14 Topological ordering · 13.15 Weighted shortest paths and Dijkstra's boundary · 13.17 Greedy algorithms · 13.18 Dynamic programming · 13.19 Algorithm selection as evidence.